

[L0-L1 核心本质与设计逻辑]

L0 一句话本质：从实模式汇编引导与内核解压自举起点出发，通过两级引导导向保护/长模式平滑过渡，利用伙伴系统的 2 的幂次分割与 SLUB 对象复用机制管理物理内存，借由红黑树 CFS 调度器平滑 CPU 算力，并通过 VFS 文件描述符分发与 Page Cache 延迟回写屏蔽底层设备差异的单体宏内核。

L1 四句话逻辑：

- 1. **引导自举与初始化** (M1) 经历 BIOS/UEFI 引导、解压内核、从汇编 head_64.S 建立临时页表过渡到 C 语言 start_kernel() 全局系统初始化。
- 2. **中断陷阱与 CFS 调度** (M2) 依托硬件中断描述符表 (IDT)，通过 common_interrupt 封装寄存器上下文，并利用以红黑树为核心的 CFS 调度进程。
- 3. **伙伴系统物理分配** (M3) 将物理内存按 Zone (DMA/Normal/HighMem) 管理，以 2 的幂次 Page 块为单位利用 Buddy 算法分割与归并，规避外部碎片。
- 4. **Slab 与块设备 Page Cache** (M4-M5) 在伙伴系统之上建立 SLUB 细粒度对象缓存池，通过 Page Cache 脏页环形队列及换页 LRU 算法优化磁盘 I/O。

[L2 存储引擎数据流演进拓片]

• [汇编引导 head_64.S] → [C 语言 start_kernel()] → [E820 内存布局探查] → [伙伴系统 Free List 初始化] → [SLUB 内核缓存池建立] → [CFS 调度器启动] → [VFS 虚拟文件系统挂载] → [Page Cache 读入文件] → [BIO 请求队列落盘]

[M1.1] x86 引导扇区载入与实模式向长模式切换 (Boot to Long Mode)

- **引导状态跳转：**BIOS/UEFI 读取磁盘 MBR，载入引导装载程序（如 GRUB），将内核 bzimage 读取到内存，随后跳转至 boot/header.S 汇编入口。
- **64 位长模式过渡：**执行 16 位实模式自检后，开启 CR0 寄存器的 PE 位切换到 32 位保护模式；随后建立两级临时物理映射页表，将 CR4 寄存器的 PAE 位置 1，修改 EFER MSR 寄存器，最终进入 64 位长模式。

[M1.2] head_64.S 汇编初始化与 C 语言入口 (head_64.S to start_kernel)

- **汇编段自举：**CPU 进入 64 位后，跳转到 arch/x86/kernel/head_64.S。在这里执行最初的汇编代码：清除 BSS 段、设置段寄存器、分配临时的内核栈 (Kernel Stack) 并加载正式的 GDT/IDT 指针。
- **跳转 start_kernel：**一切就绪后，通过 jmp 指令飞越物理边界，进入内核真正的 C 语言初始化总入口 start_kernel()。

[M1.3] setup_arch() 与 E820 内存布局探查 (setup_arch & E820)

- **物理内存探查：**setup_arch() 接收来自 bootloaders 的 boot_params 参数。通过读取 E820 物理内存表，掌握宿主主机内存的空闲与保留边界。
- **memblock 临时分配器：**在正式的伙伴系统建立前，使用 memblock 结构体进行粗粒度的物理内存管理与保留区划定，防止初始化阶段内核内存被自我覆盖。

[M1.4] init_IRQ() 与内核早期页表接管 (init_IRQ & Kernel Page Tables)

- **中断机制初始化：**init_IRQ() 初始化中断描述符表 (IDT)，写入内核正式的软硬件中断门 (Interrupt Gates)。
- **接管早期页表：**销毁 bootloaders 建立的临时物理页表，切换到由内核编译期静态定义的 swapper_pg_dir 页表，开启内核虚拟地址空间映射，为进程分配奠定基础。

[M2.1] IDT 中断描述符表与硬件中断物理压栈 (IDT & Interrupt Stack)

- **IDT 物理构造：**IDT 是内核维护的 256 项数组，每项 (Gate Descriptor) 包含 64 位的中断服务例程 (ISR) 入口虚拟地址及特权级。
- **CPU 硬件压栈：**当中断发生时，CPU 硬件自动暂停执行，将当前 SS、RSP、RFLAGS、CS、RIP 压入当前进程的内核栈，读取 IDT 对应项，并跳转至通用中断入口 common_interrupt。

[M2.2] syscall 硬件指令与系统调用表分发 (syscall & sys_call_table)

- **硬件指令特权转换：**x86_64 放弃了传统的 int 0x80 软件中断，改用 syscall 指令。CPU 读取 IA32_LSTAR MSR 寄存器，瞬间将 RIP 设为 entry_syscall_64 并切换特权级。
- **系统调用表分发：**在汇编入口保存寄存器后，根据 RAX 中的调用号，检索 sys_call_table 函数指针数组，跳转执行具体的内核服务。

[M2.3] 软中断 (softirq) 与工作队列 (workqueue) 底半部 (Bottom Halves)

- **中断上下文快速让出：**为了防止硬中断阻塞网卡或磁盘，内核将中断处理分为两半。顶半部只做最少寄存器操作，然后挂起软中断。
- **下半部异步处理：**1. **softirq (软中断)：**在硬中断返回前或 ksoftirqd 进程中执行，不能阻塞休眠，适合网卡收包。
2. **workqueue (工作队列)：**由内核背景线程执行，运行在进程上下文，可以安全执行 I/O 或睡眠阻塞。

[M2.4] CFS 完全公平调度器核心虚拟运行时间更新 (CFS vruntime Update)

- **虚拟时间轴递增：**CFS 调度器通过 vruntime 衡量进程获得的 CPU 资源。进程运行一段时间后，虚拟时间增加： $\Delta vruntime = \Delta physical_time \times \frac{NICE_0_LOAD}{weight}$ 。
- **红黑树动态排序：**CFS 使用红黑树 rb_root_cached 挂载所有 runnable 进程，以 vruntime 为键值排序。每次调度通过 $O(1)$ 的时间复杂度快速取走最左侧叶子节点 (rb_leftmost) 执行。

[M2.5] switch_to() 上下文切换汇编内核栈重定向 (switch_to Context Switch)

- **硬件寄存器重定向：**调度程序决定切换进程时，调用 switch_to()。
- **汇编重与栈指针：**利用汇编修改 RSP 寄存器，将其重定向到目标进程的 thread.sp（内核栈顶）。由于所有局部变量和 CPU 寄存器都在栈中，完成栈指针切换即完成了 CPU 上下文的物理转移。

[M3.1] NUMA 架构、物理 Node 与内存 Zone 划分 (NUMA Zones)

- **NUMA 物理节点：**多路 CPU 下，物理内存被划分为多个 Node（结构为 pg_data_t），靠近各自 CPU 以减少总线访问开销。
- **Zone 物理分层：**每个 Node 内部根据硬件限制划分为：
1. **ZONE_DMA：**物理内存前 16MB，用于兼容老式 DMA 硬件。
2. **ZONE_NORMAL：**16MB 至 4GB，内核可以直接映射访问的主物理区域。
3. **ZONE_HIGHMEM：**4GB 以上（32 位系统下无法直接映射的区域，64 位下已基本不存在）。

[M3.2] struct page 物理页帧元数据映射机制 (struct page Layout)

- **全局页描述符数组：**内核在初始化时，为系统内所有 4KB 物理页框分配一个 struct page 描述符，集中存放在 mem_map 数组中。
- **映射计数控制：**struct page 不包含任何文件数据，仅用 _refcount（内核引用计数）和 _mapcount（用户态页表映射次数）控制页面的生命周期。当 _refcount 归 0 时，页面被释放回伙伴系统。

[M3.3] 伙伴系统 (Buddy System) 2 的幂次物理分割算法 (Buddy Allocator)

- **幂次空闲链表数组：**伙伴系统维护一个从 0 到 10（即 1 页到 1024 页）的 free_area 链表数组。
- **异或比对与拆分归并：**
- **分配：**申请 Order 级页块时，若对应链表为空，则向更高级链表申请，并将得到的大块对半“拆分 (Split)”。
- **释放：**释放页块时，计算伙伴块地址 $Partner = Block \oplus (Order \ll PAGE_SHIFT)$ ，若伙伴块也空闲，则将其“合并 (Merge)”并提升到上一级 Order 链表。

[M3.4] 内存异步整理 (Migration & Compaction) 机制 (Memory Compaction)

- **内存碎片化痼疾：**长时间运行后，频繁分配释放会导致物理内存中充斥着散落的已分配页，无法找到 Order-9（2MB）等连续大块页。
- **双向扫描平移整理：**kcompactd 线程从 Zone 两端向中间扫描；一端扫描可移动的 active 页面，另一端扫描空闲块。将 active 页面整体平移复制到空闲块中，并在页表中重定向，物理拼接出大片连续的空闲空间。

[壹] Linux 内核虚拟化、开发与物理性能折衷防范矩阵 (Linux Kernel Core Trade-offs)

• **textbf{SLUB 内存分配会尽量将物理空间紧密排列，以防止内核产生空间碎片}：**textbf{为了消除 CPU 核心之间的 Cache Line 伪共享 (False Sharing)，SLUB 分配器在物理内存上会故意按照 CPU L1 Cache Line 大小（通常是 64 字节）对齐。这会导致小于 64 字节的对象也会占用 64 字节，产生一定的物理内存浪费。} → 采用 Cache Line 物理对齐设计。通过在 kmem_cache_create 中使用 SLAB_BROADCAST 标志，确保频繁访问的对象在物理上独占 Cache Line，虽然牺牲了微小的内存利用率，但极大地换取了多核并发时的 CPU 吞吐。

• **textbf{CFS 完全公平调度器会绝对公平地平摊 CPU 时间给所有活跃的线程}：**textbf{绝对的公平会导致严重的“线程饥饿”与高频上下文切换开销 (Context Switch Overheads)。}如果让大量短任务和长计算任务频繁打断切换，CPU 将把大部分时间浪费在保存恢复寄存器和刷新 MMU 页表上。 → 引入调度最小粒度 sched_min_granularity_ns。CFS 保证进程被调度后，必须至少连续运行此最小时间。防止因频繁计算虚拟时间 vruntime 产生过度切换，在公平性与切换开销之间达成折衷。

• **textbf{为了加速磁盘文件读取，我们应该把 vm.dirty_background_ratio 阈值设得无限高}：**textbf{脏页阈值过高会导致大量的脏数据滞留在内存中。当系统内存面临回收或主动执行 sync 时，海量脏页集中回写，会瞬间打满块设备 I/O 通道，引发严重的系统级 Stall 停摆。} → 合理调优写回参数。在生产高吞吐服务器上，通常建议将 vm.dirty_background_ratio 限制在 5%-10%，并结合 vm.dirty_ratio 设为 20% 左右，以保持内核 pdflush 背景回写线程高频平滑写入。

